

Equality of `svp_smuce_cpp()` and `stepR::stepFit()`

12 September 2026

This note gives the exact parameterization under which the SMUCE routine `svpChange2::svp_smuce_cpp()` and `stepR::stepFit()` solve the same problem. The comparison concerns independent Gaussian observations with a **known, constant** standard deviation $\sigma > 0$ and the complete system of subintervals.

The common multiscale constraint

Let $y_1, \dots, y_n \in \mathbb{R}$, and let $0 = t_0 < t_1 < \dots < t_K = n$ define the segments $I_k = \{t_{k-1} + 1, \dots, t_k\}$. For a subinterval $J = \{u, \dots, v\}$ of length $m = v - u + 1$ and a candidate constant mean θ , the Gaussian log-likelihood ratio is

$$T_J(\theta) = \frac{m(\bar{y}_J - \theta)^2}{2\sigma^2}.$$

SMUCE accepts (J, θ) at threshold q when

$$\sqrt{2T_J(\theta)} - \sqrt{2\log(en/m)} \leq q.$$

Equivalently, θ must belong to the interval

$$[\bar{y}_J - r_m(q), \bar{y}_J + r_m(q)], \quad r_m(q) = \frac{\sigma}{\sqrt{m}} \left(q + \sqrt{2\log(en/m)} \right).$$

Hence a candidate segment I is valid if and only if

$$\Theta_I(q) = \bigcap_{J \subseteq I} [\bar{y}_J - r_{|J|}(q), \bar{y}_J + r_{|J|}(q)] \neq \emptyset.$$

In `stepFit()`, a scalar `q` with `penalty = "sqrt"` is internally converted, for every length m , to the likelihood-ratio bound

$$c_m(q) = \frac{1}{2} \left(q + \sqrt{2\log(en/m)} \right)^2.$$

More specifically, $c_m(q)$ is an upper bound on the statistic $T_J(\theta)$ for every subinterval J of length m and candidate mean θ :

$$T_J(\theta) \leq c_m(q).$$

Indeed, the SMUCE inequality gives $\sqrt{2T_J(\theta)} \leq q + \sqrt{2\log(en/m)}$; squaring yields the displayed bound (with $q \geq 0$). Thus $c_m(q)$ is not a global bound independent of scale: it is the length-specific threshold applied separately to every J .

In `svp_smuce_cpp()`, the lower and upper endpoints of the same intersection $\Theta_I(q)$ are updated directly using $r_m(q)$. Thus the two routines have exactly the same valid segments when they receive the parameter values listed below.

The common optimization problem

Write $\Theta_{I_k}(q) = [L_k, U_k]$ for a valid segment. Its constrained Gaussian maximum-likelihood estimate is

$$\hat{\theta}_k = \text{proj}_{[L_k, U_k]}(\bar{y}_{I_k}) = \min\{\max(\bar{y}_{I_k}, L_k), U_k\}.$$

Both routines solve the lexicographic problem

$$(\hat{t}, \hat{\theta}) \in \arg \text{lex min}_{\substack{0=t_0 < \dots < t_K=n, \\ \theta_k \in \Theta_{I_k}(q)}} \left(K, \sum_{k=1}^K \frac{1}{\sigma^2} \sum_{i \in I_k} (y_i - \theta_k)^2 \right).$$

They therefore first minimize the number K of valid constant segments and, among those partitions, minimize the constrained Gaussian residual cost. `stepFit()` expresses the second coordinate as a negative log-likelihood; its factor $1/2$ and data-only additive constant do not change the minimizer. Consequently the two routines have the same feasible set and the same lexicographic minimizer set. If this set contains one segmentation, the returned segment endpoints are identical. If several segmentations attain the same lexicographic minimum, each implementation may return a different member of that common set according to its deterministic tie rule; this is a tie, not a disagreement about the optimization problem.

How `svp_smuce_cpp()` implements SVP

The implementation follows the SVP formulation. Write $y_{a:b} = (y_{a+1}, \dots, y_b)$, with $0 \leq a < b \leq n$, and let $\tau = (0 = \tau_0 < \dots < \tau_K = n)$ be a segmentation. For a candidate segment, define

$$\Theta_{a:b}(q) = \bigcap_{a \leq u < v \leq b} [\bar{y}_{u:v} - r_{v-u}(q), \bar{y}_{u:v} + r_{v-u}(q)].$$

The segment is valid precisely when $\Theta_{a:b}(q) \neq \emptyset$. The code stores only its endpoints, $L_{a:b} = \inf \Theta_{a:b}(q)$ and $U_{a:b} = \sup \Theta_{a:b}(q)$. When the right endpoint is extended, the intersection is updated with the new subinterval constraints:

$$\begin{aligned} L_{a:b} &= \max \left\{ L_{a:b-1}, \max_{a \leq u < b} (\bar{y}_{u:b} - r_{b-u}(q)) \right\}, \\ U_{a:b} &= \min \left\{ U_{a:b-1}, \min_{a \leq u < b} (\bar{y}_{u:b} + r_{b-u}(q)) \right\}. \end{aligned}$$

Thus validity is available as soon as $L_{a:b} \leq U_{a:b}$. The mean used by the secondary criterion is the projection

$$\hat{\theta}_{a:b} = \text{proj}_{[L_{a:b}, U_{a:b}]}(\bar{y}_{a:b}), \quad C_{a:b} = \frac{1}{\sigma^2} \sum_{i=a+1}^b (y_i - \hat{\theta}_{a:b})^2.$$

For each prefix $y_{0:t}$, the dynamic-programming state is the SVP pair $R_t = (K_t, Q_t)$: K_t is the smallest number of valid segments and Q_t is the smallest total constrained cost among segmentations attaining K_t . With lexicographic order $\prec$, the recursion is

$$R_t = \text{lex min}_{0 \leq a < t: \Theta_{a:t}(q) \neq \emptyset} \{ R_a + (1, C_{a:t}) \}, \quad R_0 = (0, 0).$$

The predecessor p_t is stored whenever a candidate improves the current pair. Starting from $t = n$, repeated assignments $t \leftarrow p_t$ recover the endpoints in reverse order; the returned vector contains the internal changepoints followed by n . Hence the routine is an exact SVP dynamic program, rather than a greedy scan.

For numerical stability, the implementation centers observations as $z_i = y_i - y_1$ before constructing prefix sums. Means and residual sums of squares are computed from these centered sums, while validity intervals are translated back by y_1 . This preserves translation invariance and avoids cancellation when the observations have a large common offset. The R routine `svp_smuce()` uses the same recurrence as a direct reference implementation.

Why the compiled implementation is $O(n^2)$

The key optimization is that the interval for $y_{a:b}$ is obtained from the two already available neighbouring intervals, rather than by rescanning all subintervals. If $\mathcal{I}_{a:b} = [L_{a:b}, U_{a:b}]$ denotes the current intersection and $\mathcal{J}_{a:b}$ the constraint from the whole segment $[a+1, b]$, then

$$\mathcal{I}_{a:b} = \mathcal{I}_{a+1:b} \cap \mathcal{I}_{a:b-1} \cap \mathcal{J}_{a:b}.$$

For a fixed right endpoint b , `svp_smuce_cpp()` visits $a = b, b-1, b-2, \dots, 0$ and updates the lower and upper endpoints with this constant-work recurrence. There are $n(n+1)/2$ pairs (a, b) , so all validity intervals and all dynamic-programming candidates are processed in $O(n^2)$ time. Prefix sums make each segment mean and constrained residual cost $O(1)$; rolling arrays retain only the previous and current right-endpoint columns, giving $O(n)$ working memory (apart from the predecessor and DP vectors). Invalidity is monotone when a segment is extended, so the inner scan can stop as soon as $L_{a:b} > U_{a:b}$. The R reference currently spells out the subinterval intersection directly and is therefore less optimized, while the compiled routine uses the two-neighbour recurrence above.

Parameterization required for equality

Mathematical choice	<code>svpChange2</code>	<code>stepR</code>
observations	<code>y</code>	<code>y</code>
Gaussian model	built in	<code>family = "gauss"</code>
known noise scale	<code>sigma2 = σ^2</code>	<code>sd = σ</code>
SMUCE threshold	<code>q = q</code>	<code>q = q</code>
intervals scanned	all subintervals	<code>intervalSystem = "all",</code> <code>lengths = seq_len(n)</code>
scale correction	built in	<code>penalty = "sqrt"</code>
comparable output	returned integer vector	<code>as.integer(fit\$rightIndex)</code>

The corresponding calls are:

```
n <- length(y)

fit_svp <- svpChange2::svp_smuce_cpp(
  y = as.double(y), q = q, sigma2 = sigma^2
)

fit_stepR <- stepR::stepFit(
  y = as.double(y), q = q,
  family = "gauss", sd = sigma,
  intervalSystem = "all", lengths = seq_len(n),
  penalty = "sqrt", confband = FALSE, jumpint = FALSE
)

identical(fit_svp, as.integer(fit_stepR$rightIndex))
```

Here `q` is the SMUCE threshold itself—it is neither `q^2` nor `alpha`—and both endpoint vectors include the final index n . Explicitly requesting all lengths is essential for equality when $n > 1000$, because `stepFit()` otherwise defaults to a dyadic interval system. The argument mapping was checked with `stepR` 2.1-11 and `svpChange2` 0.1.0.

Reference. Frick, K., Munk, A. and Sieling, H. (2014). Multiscale change-point inference. *JRSS B*, 76(3), 495–580.